

# Agentic Scheduling with Gemini-Driven Partition Reasoning and Local Granularity Refinement

MLSys 2026 Scheduling Contest — Track B

Hoyun Youm  
Yonsei University  
hoyun@yonsei.ac.kr

Chan Lee  
Yonsei University  
chan.lee@yonsei.ac.kr

## Introduction

The contest asks the participant to schedule a memory-bounded DAG of MatMul and Pointwise operators on a two-level memory hierarchy, using only the Gemini API for reasoning. A schedule makes four interacting decisions per subgraph: *subgraphing*, *retention*, *granularity*  $[w, h, k]$ , and *traversal order*. The structural observation driving our design is that partitioning dominates the complexity, while granularity and traversal can be searched locally with small enumerations.

Our system splits the two classes of reasoning between the LLM and deterministic code. Gemini handles the combinatorial, semantically rich task of proposing a partition and retention plan; Python code handles granularity and traversal search, where exhaustive enumeration over a small candidate set is both feasible and precise. The agent runs a best-of- $N$  loop over Gemini attempts with a cost-feedback prompt, and closes the loop with a long final grain-refinement pass over the best partition found.

## Challenges for an LLM Partition Designer

Three properties of the partition space are difficult for a language model. (1) *Irregular cost*: small partition edits cause large discontinuous jumps in memory traffic or feasibility, so the LLM cannot reliably improve a partition by incremental rewrites. (2) *Hidden arithmetic*: whether an intermediate becomes ephemeral depends on consumer sets across the whole subgraph; whether a given  $[w, h, k]$  fits fast memory depends on the per-role working-set formula (LHS resident  $h \times K$  for chain-upstream MM, etc.). These are tedious and error-prone for a text-generating model. (3) *Stochastic sampling*: the same prompt yields different partitions across runs.

We address (1) and (2) by pre-digesting the graph for the LLM and by letting deterministic code enforce every hard constraint; (3) with a best-of- $N$  harness and a cost-feedback log threaded through the prompts.

## System Overview

Each invocation proceeds through four stages.

1. *Safety-net fallback*. Emit a 1-op-per-subgraph solution with an analytic grain picker and write it to disk before any Gemini call. Guarantees a valid output if later stages fail.
2. *Fallback refinement*. Run 2-stage grain search over the fallback partition using  $\sim 30\%$  of the time budget. This alone closes 5–15% of the gap on most benchmarks.
3. *Best-of- $N$  Gemini loop*. Three attempts with distinct (strategy, temperature) pairs: (rich, 0.25), (minimal, 0.70), (rich, 0.55). Each attempt proposes a partition and retention plan; our

code runs grain search over it and keeps the best valid solution. A compact cost-feedback summary of prior attempts is threaded into the next prompt so Gemini can reason about what structural change to try next.

4. *Final refinement*. After the Gemini loop, spend all remaining budget on a high-quality grain search over the best partition seen. During Gemini attempts the per-subgraph grain budget is necessarily short (2–3 s for the 100-op benchmark), often too short to reach the optimum; the final pass gives each subgraph 10–15 s of dedicated search.

## Prompt Design

The system prompt encodes every evaluator rejection path as six invariants: coverage and producer-before-consumer order; retention validity (only op-produced tensors can be retained, lifetime one step); the ephemeral-with-external-consumer rule; the unified execution grid, including the clarification that ephemeral intermediates are exempt from the shape-matching requirement; the MM-chain fusion rules under full- $K$  vs. split- $K$ ; and the per-role working-set formula (LHS-head resident  $h \times K$ , standalone MM LHS  $h \times k_{\text{eff}}$ , pointwise  $w \times h$ , etc.). Encoding these invariants in the prompt means Gemini knows, before proposing anything, exactly which configurations will be rejected.

The user prompt carries a client-side pre-analysis of the graph so Gemini does not have to scan the raw JSON to find structure: output-shape buckets, shared graph-input clusters, pre-checked fusion-candidate edges, and *triple-fusion candidates* — ordered triples of the form (pointwise, matmul, pointwise) where the matmul’s two adjacent intermediates can both be made ephemeral. Each triple is annotated with a producer-shape *masking warning* when an earlier MM producing the triple’s input has a larger output shape, to prevent Gemini from extending the fusion backwards into a large-shape MM — which would otherwise force the subgraph to tile at the small output’s grid and leave the large MM mostly masked.

## Triple Fusion with Ephemeral MatMul Operands

The single biggest improvement came from Gemini learning to fuse a three-operator chain: an upstream pointwise whose output feeds the MatMul’s LHS, the MatMul itself, and a downstream pointwise that consumes the MatMul’s output. Because both of the MatMul’s adjacent tensors are produced *and* consumed entirely inside the subgraph (and neither is listed in `tensors_to_retain`), both become ephemeral by definition. Crucially, the usual dominant working-set term — the MatMul’s resident LHS of size  $h \times K$  — **disappears** because the LHS is no longer a resident input: it is

produced tile-by-tile by the upstream pointwise and consumed immediately by the MatMul.

For a large- $K$  operator (e.g.  $K = 4096$  with the per-benchmark capacity of a few hundred thousand cells), this shrinks the working set enough to permit full- $K$  execution ( $nk = 1$ ), eliminating the split- $K$  reload penalty that dominates the per-op baseline. Once the system prompt includes this working-set calculation explicitly, Gemini identifies the triples in the graph and proposes the fusion directly.

## Grain Search

Candidates are a union of powers-of-two, divisors of the output dimension, the native granule, and a *tile-count sweep* so non-standard values are reachable (e.g.  $w = 103$  for  $W_{\text{out}} = 4096$  with 40 tiles, where neither 103 nor its complement is a power of two). Snake traversals (row-snake and column-snake) are only useful when the reused operand matches the unchanging tile axis, and the benefit is multiplicative on top of the row-major cost, so we use a two-stage search: Stage 1 enumerates all candidates under row-major

traversal and keeps the top-10 by cost; Stage 2 tries the two snake variants on that shortlist. This is roughly  $3\times$  faster than evaluating all three traversals at every candidate — necessary to keep the 103-op benchmark inside the time budget.

The tile grid on which each snake permutation is built is derived from the *non-ephemeral* outputs of the subgraph, matching the evaluator’s own grid definition. A subgraph containing an ephemeral  $4096\times 1024$  intermediate and a non-ephemeral  $1024\times 1024$  output tiles over a  $1024\times 1024$  grid, not the  $4096\times 1024$  one, so the snake permutations must have length  $\lceil 1024/w \rceil \cdot \lceil 1024/h \rceil$ .

## Reproducibility

Submission contains `agent.py` (entry point), `prompts/` (system and repair templates), and `requirements.txt` (google-genai). Invocation is the contest-specified `python3 agent.py <problem.json> <output.json>`; the agent expects `GOOGLE_API_KEY` in the environment and opens no other network connection. The `TEAMKOREA_BUDGET_S` environment variable (default 540 s) leaves a 60-second margin inside the organiser’s 600-second per-benchmark timeout.